Recados importantes

o que fazer · quando chega · como usar o chat · onde roda

ação sua · entrega para você

1 — ÚLTIMA CHANCE — **Hoje, até 12:00** — comprar a gravação

2 — AÇÃO — **Dia 01, tarde** — assinar a lista

3 — AÇÃO — **Dia 02, tarde** — assinar a lista

4 — ENTREGA — **Até 23/10** — certificado emitido e enviado

5 — ENTREGA — **Até 26/10** — gravação na Kiwify

Certificado

emitido pela Faculdade Full Cycle

Dia 02

Lista de presença · Dia 01

Nome	Assinatura
seu nome	ok
...	
...	

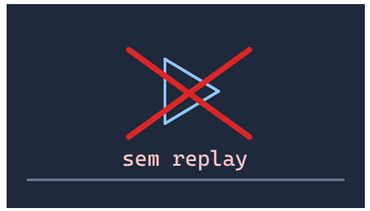
2 3 a lista circula à tarde nos dois dias
(o instrutor relembra na hora)

sem assinatura nos dois dias
⇒ sem certificado

4 emissão e envio até 23/10

Gravação

o evento não tem replay



comprou

recebe até 26/10
na Kiwify

Dúvidas e chat

três estados ao longo do evento

Durante a explicação

chat desligado — off

dispersa quem está acompanhando na máquina

Durante o evento

só dúvidas de conteúdo

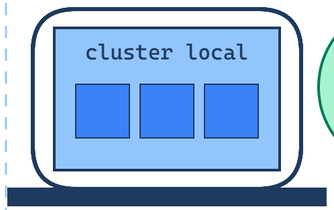
dúvida de conteúdo — ok

No final

espaço aberto
para as demais dúvidas

Onde isso roda

tudo na sua máquina



sem conta de cloud
sem custo
sem quota

máquina do aluno

Kubernetes é agnóstico:
o que se constrói aqui
vale lá

cluster gerenciado

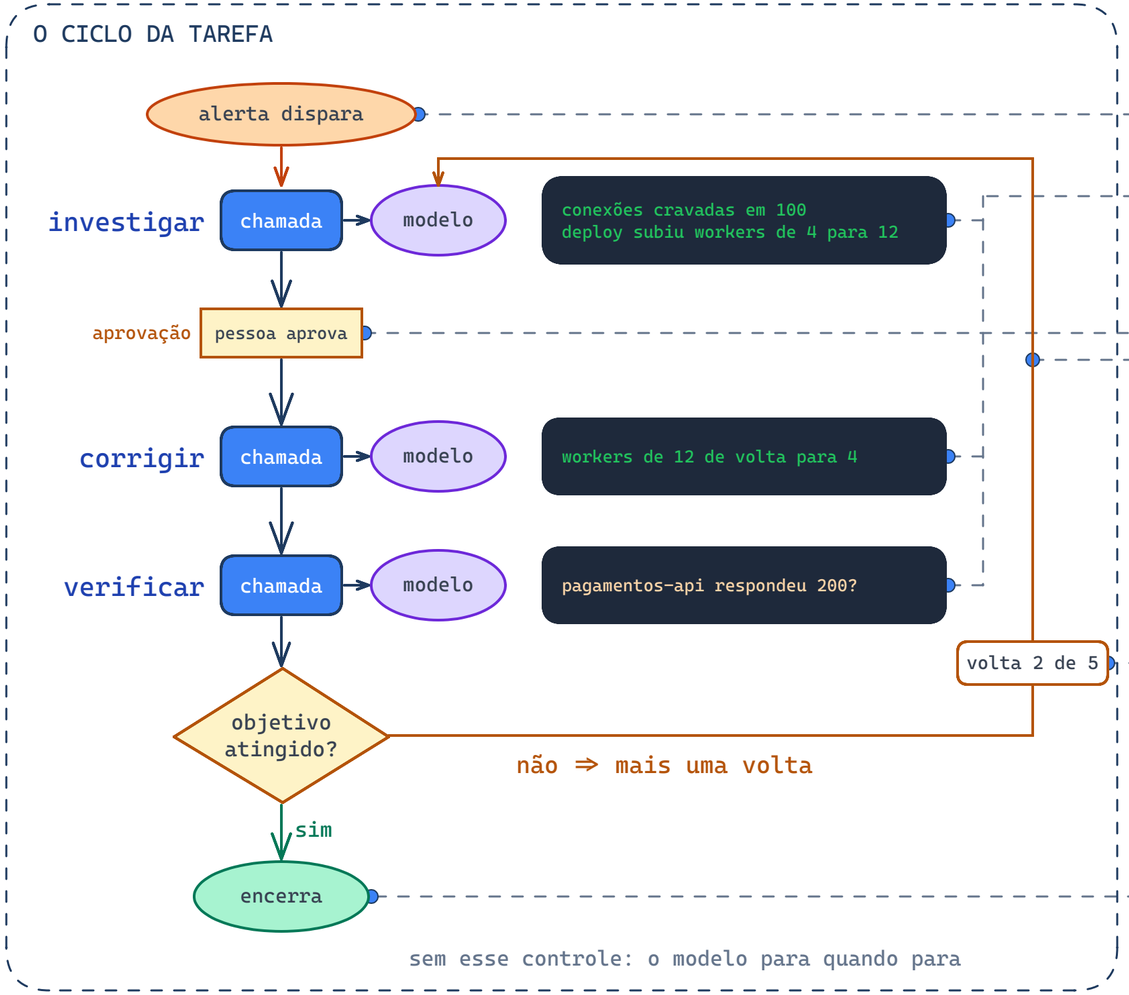
o que muda fora do cluster local:
fim de domingo, na correção

Patrocínio: Pós-Graduação em AIOps e Engenharia Cloud

Engenharia de loop

quem escreve o loop controla o ciclo, não o modelo

Loop
Harness
Contexto
Prompt



o que quem escreve o loop controla

quando começa
⇒ o alerta do pagamentos-api dispara a 1ª volta

o que cada volta faz
⇒ investigar · corrigir · verificar

onde o humano entra
⇒ aprova a correção antes de aplicar

quando dá outra volta
⇒ o objetivo ainda não foi atingido

até quantas voltas
⇒ no máximo 5, depois passa para uma pessoa — máx 5

quando para
⇒ o serviço voltou a responder 200

Engenharia de loop

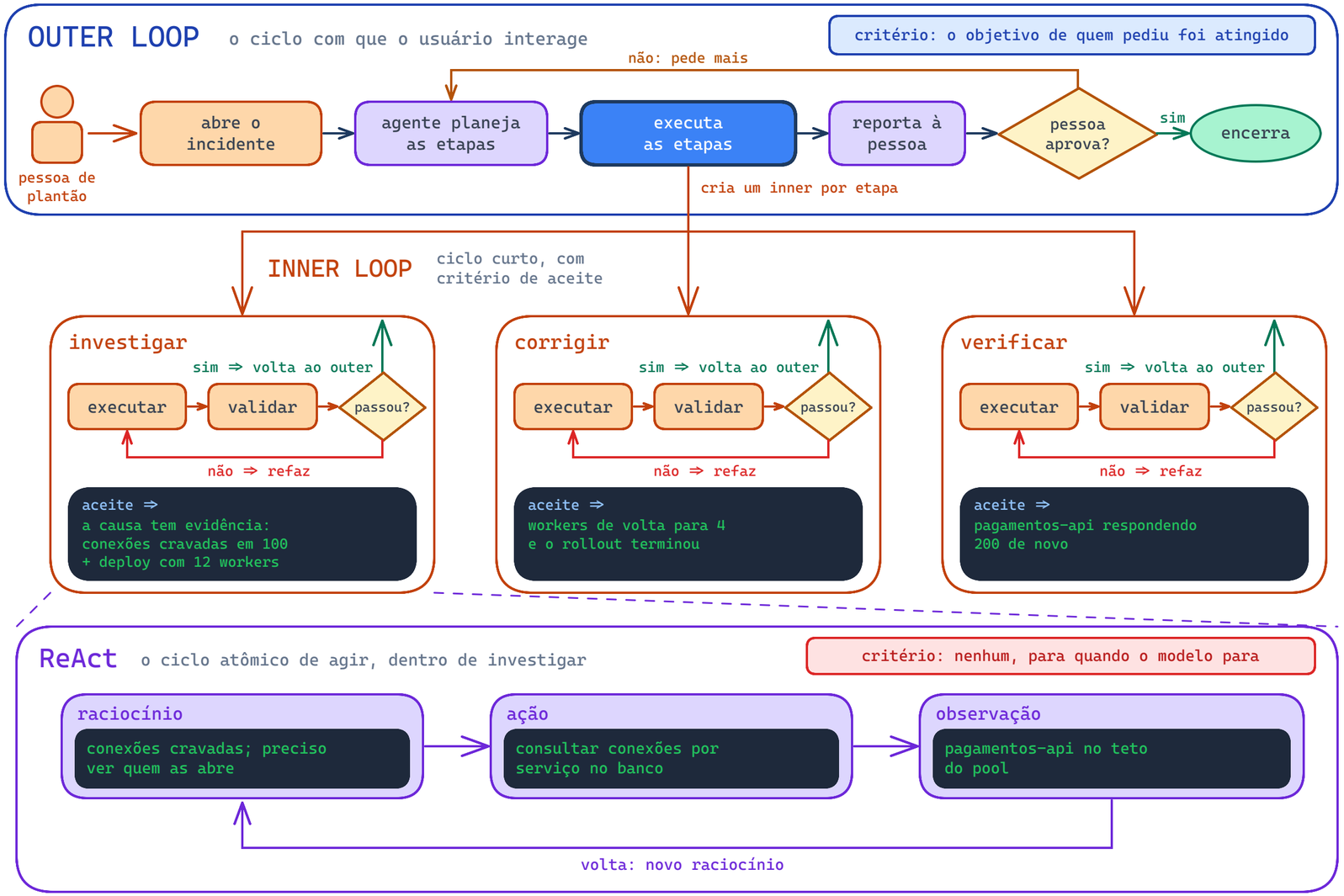
O que é
⇒ desenhar o ciclo e quem decide que acabou

O que decide
⇒ o critério de parada não o momento em que a IA para

Quanto dura
⇒ sobrevive à pessoa que escreveu

Três ciclos, três critérios

o outer cria inner loops; cada inner age em ciclos de ReAct



critério por nível

outer

objetivo de quem pediu

sem ele ⇒ roda em círculo sem fechar

inner

critério de aceite da etapa

sem ele ⇒ desiste com a etapa pela metade

ReAct

nenhum

sozinho ⇒ confunde parar com concluir

pergunta em aberto

como um agente sabe que o incidente acabou?

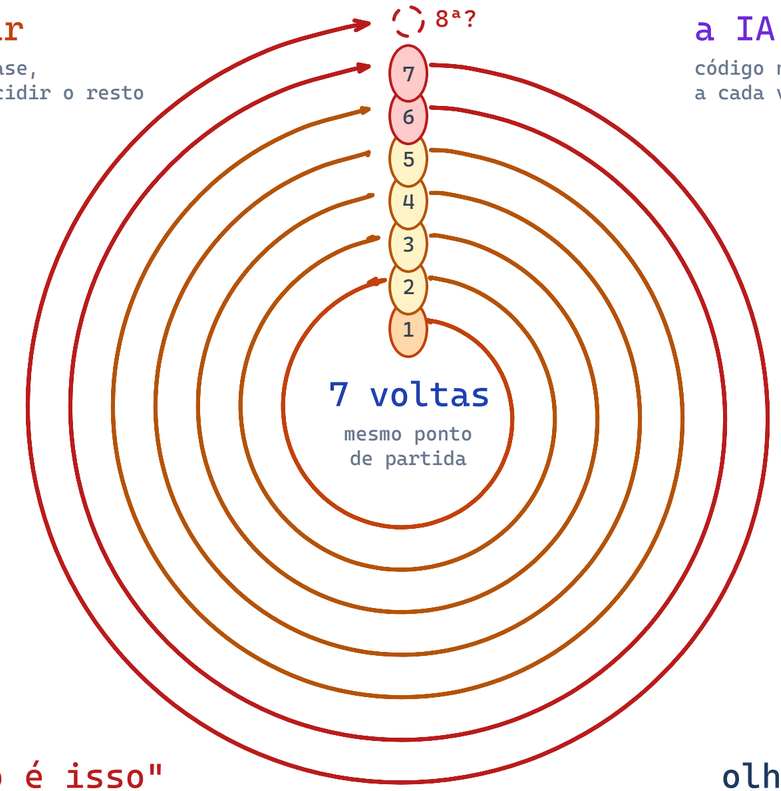
Sete mensagens para um reembolso

cada volta corrige o que ninguém decidiu antes de pedir

pedir → a IA gera → olhar → "não é isso" → pedir de novo

pedir
uma frase,
sem decidir o resto

a IA gera
código novo
a cada volta



"não é isso"
a correção vira
a próxima mensagem

olhar
roda e confere
no olho

sessão · pagamentos-api — aberta

1. Você: Adiciona reembolso parcial na API de pagamentos.
   - IA: Pronto. Criei POST /pagamentos/{id}/reembolso recebendo o valor.
2. Você: Não pode reembolsar mais que o valor pago. — decisão que não estava na frase
3. Você: E se já teve reembolso antes? A soma não pode passar do total. — decisão que não estava na frase
4. Você: Pagamento estornado por inteiro tem que recusar. — decisão que não estava na frase
5. Você: Só quem é do financeiro pode pedir. — decisão que não estava na frase
6. Você: Agora quebrou o estorno total que já existia. — efeito colateral da volta anterior
7. Você: Por que o valor está arredondando errado? — efeito colateral da volta anterior

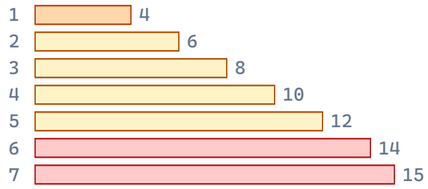
Você: escreva a próxima correção …

o código cresce

reembolso.py depois da volta 7

```
1  @post("/pagamentos/{id}/reembolso")
1  def reembolsar(id, valor):
1    pg = buscar(id)
2    if valor > pg.valor:
2      return recusar()
3    total = pg.reembolsado + valor
3    if total > pg.valor: recusar()
4    if pg.status == "estornado":
4      return recusar()
5    if not usuario.financeiro:
5      return negar()
6    if valor == pg.valor:
6      return estorno_total(pg)
7    valor = Decimal(str(valor))
1    pg.reembolsar(valor)
```

linhas de código ao fim de cada volta



o código cresce mais rápido que o entendimento dele

a especificação existia, picotada em 7 mensagens · ao fechar a sessão, ela vai junto

1 · reembolso parcial | 2 · até o valor pago | 3 · soma até o total | 4 · estornado recusa | 5 · só financeiro | 6 · estorno total intacto | 7 · arredondamento → sessão fechada

a próxima começa do zero

Onde a decisão mora

o mesmo agente, a mesma feature, dois caminhos

o mesmo agente

VIBE CODE

pedir → olhar → corrigir → de novo ×7

mensagem

2 — não pode passar do valor pago → agente ajusta

3 — a soma dos reembolsos não pode passar do total → agente ajusta

4 — estornado por inteiro: recusa → agente ajusta

5 — só o financeiro pode pedir → agente ajusta

… e segue até a mensagem 7, uma decisão por mensagem

as decisões moram em

a conversa → fecha com a sessão

IA COM ENGENHARIA

especificar → construir → conferir contra o especificado

escrito antes de construir

no máximo 1 correção pontual

documento

Reembolso parcial

- a soma dos reembolsos nunca passa do valor pago
- pagamento estornado por inteiro não aceita reembolso
- só o perfil financeiro pode pedir

Aceite: pagamento de R$ 100 com R$ 70 já reembolsados, pedido de R$ 40 é recusado

Fora: reembolso em outra moeda

o critério vem daqui

uma volta, com as 4 decisões já escritas

as decisões moram em

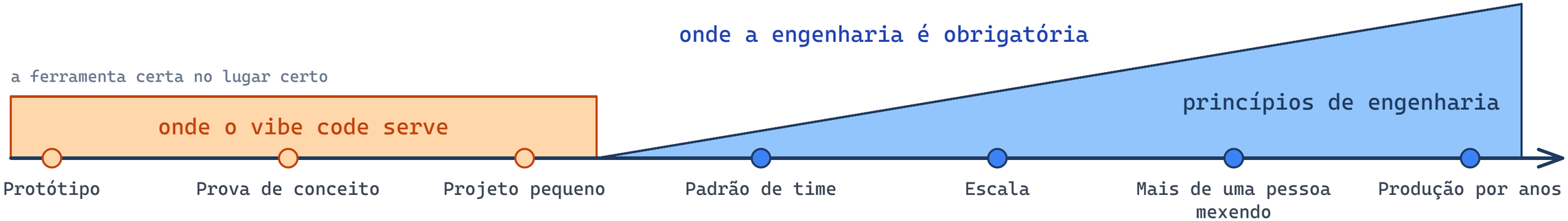
o documento → sobrevive à sessão

onde a decisão mora

7 mensagens	1 documento + 1 conferência
a construção e o critério na mesma conversa	a construção se delega · o critério não

Protótipo ou produção

um espectro, não um veredito



funciona?

pagamentos-api

testar com o financeiro se o reembolso parcial faz sentido

tentativa e erro responde:
roda, mostra, ajusta

outro dev entende?

pagamentos-api

outra pessoa mexe no reembolso sem quebrar o estorno total

aguenta o dobro?

pagamentos-api

fim de mês: o dobro de pedidos de reembolso

por que foi feito assim?

pagamentos-api

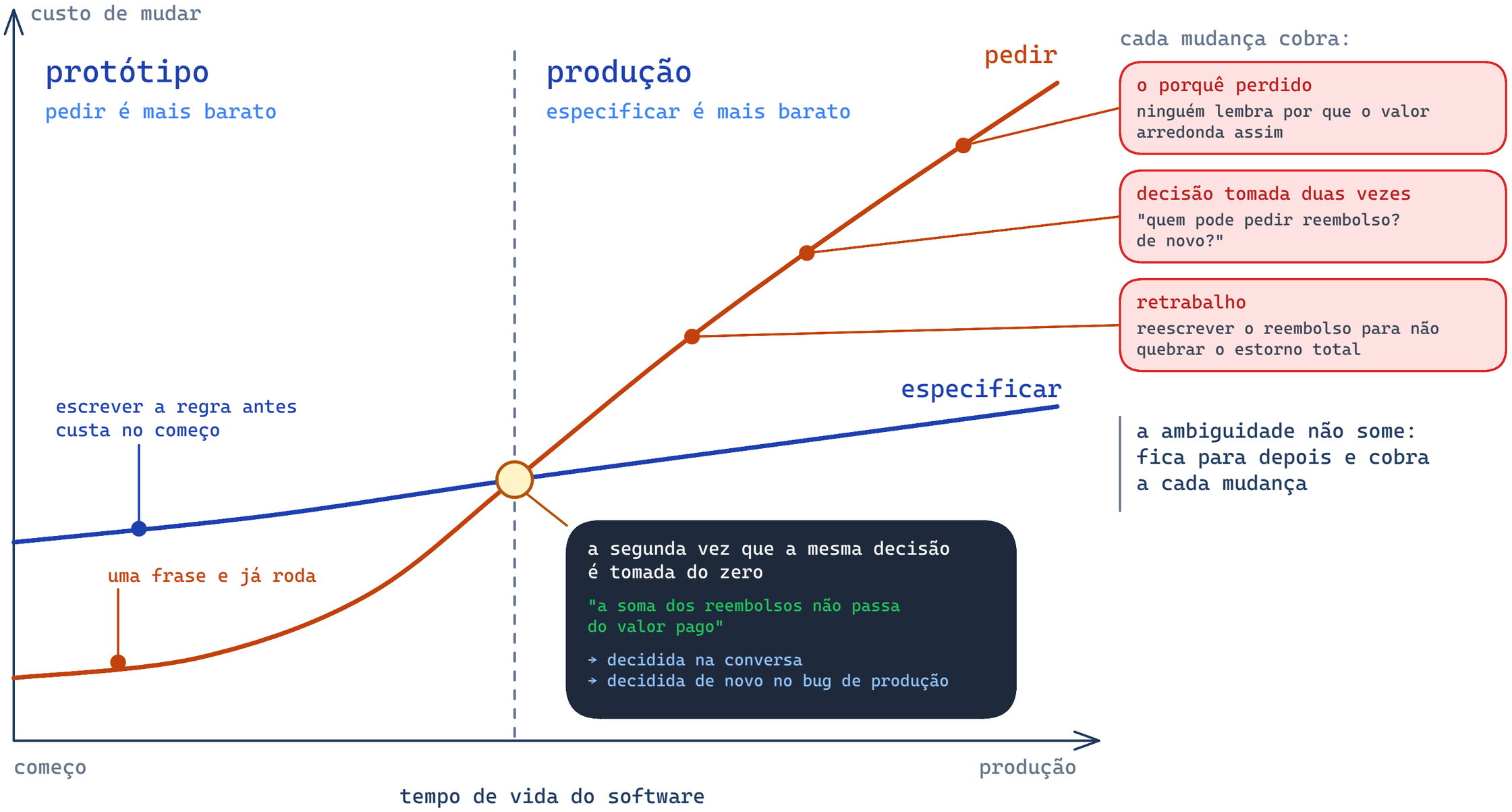
a auditoria pergunta por que o reembolso de março foi recusado

tentativa e erro não responde:
cada pergunta pede um princípio de engenharia

---

software de produção pede a IA dentro de um processo de desenvolvimento e entrega

Onde pedir passa a custar mais



cada mudança cobra:

o porquê perdido
ninguém lembra por que o valor arredonda assim

decisão tomada duas vezes
"quem pode pedir reembolso? de novo?"

retrabalho
reescrever o reembolso para não quebrar o estorno total

a ambiguidade não some: fica para depois e cobra a cada mudança

a segunda vez que a mesma decisão é tomada do zero

"a soma dos reembolsos não passa do valor pago"

→ decidida na conversa
→ decidida de novo no bug de produção

Cada documento responde uma pergunta

acima da linha, o que o sistema faz · abaixo, com o que ele é feito · a spec atravessa as duas rumo ao código

COMPORTAMENTO — o que o sistema faz

PRD — o quê e por quê?

comportamento · regra de negócio
e exceções · critério de aceite
o que está fora

● muda quando o negócio muda

```
PRD · Reembolso parcial
Regra:           a soma dos reembolsos de um pagamento
                 nunca passa do valor pago.
Exceção:         pagamento estornado por inteiro
                 não aceita reembolso.
Aceite:          pagamento de R$ 100 com R$ 70
                 reembolsados; pedido de R$ 40
                 é recusado.
Fora de escopo:  reembolso em outra moeda.
```

regra

SPEC

o que o agente constrói
e como se confere

atravessa as duas camadas:
usa a regra e a decisão

```
Requisito: recusar reembolso
acima do saldo

Dado     um pagamento de 10000 centavos
         com 7000 reembolsados
Quando   chega um pedido de 4000
Então    responde 422 e nada é gravado
```

deriva dos outros
é a que vira código

código

FRONTEIRA — a regra de negócio sobrevive à troca de stack · o contrário nunca

o PRD referencia o técnico, não o dirige

! decisão técnica dentro do PRD: o erro mais comum e o mais caro

TÉCNICO — com o que ele é feito

design doc · a família dos documentos técnicos

TRD — com o quê e como, no projeto inteiro?
stack · arquitetura · requisitos não-funcionais · padrões

```
TRD · pagamentos-api
Stack:           Python 3.12 · Postgres
Padrão:          operação financeira grava um movimento;
                 nunca altera o saldo direto.
Não-funcional:   o mesmo pedido enviado duas vezes
                 gera um reembolso só.
```

● muda quando a stack muda

ADR — por que escolhemos isto?
uma decisão: contexto · alternativas · consequências · status

```
ADR 007 · Valor em centavos
Status:                aceito · substitui o ADR 003
Contexto:              arredondamento com decimal gerou
                       diferença de 1 centavo no estorno.
Decisão:               guardar valor como inteiro,
                       em centavos.
Alternativa descartada: decimal com duas casas.
Consequência:          toda entrada e saída converte
                       na borda.
```

centavos

● não se edita: um ADR novo substitui o antigo

A régua ganha um degrau

de baixo para cima, cada camada dura mais que a anterior

novo · Spec · Skill · Prompt

permanência

o que guarda

no reembolso do pagamentos-api

quando some

além do código

Spec — novo degrau

spec

1. a decisão com a razão
2. o critério de aceite
3. o que está fora

spec do reembolso parcial

1. Requisito: recusar reembolso acima do saldo
   Razão: o financeiro não devolve mais do que recebeu
2. Dado 7000 de 10000 centavos já reembolsados
   Quando chega pedido de 4000 · Então recusa
3. Fora: reembolso em outra moeda

continua valendo se o código for reescrito

projeto

Skill

projeto/
- skills
- código

o padrão do projeto

skill do pagamentos-api

operação financeira grava um movimento

nunca altera o saldo direto

quando o projeto acaba

sobrevive

a sessão fecha aqui

morre

sessão

Prompt

o pedido

mensagem

> Adiciona reembolso parcial na API de pagamentos.

ao fechar a sessão

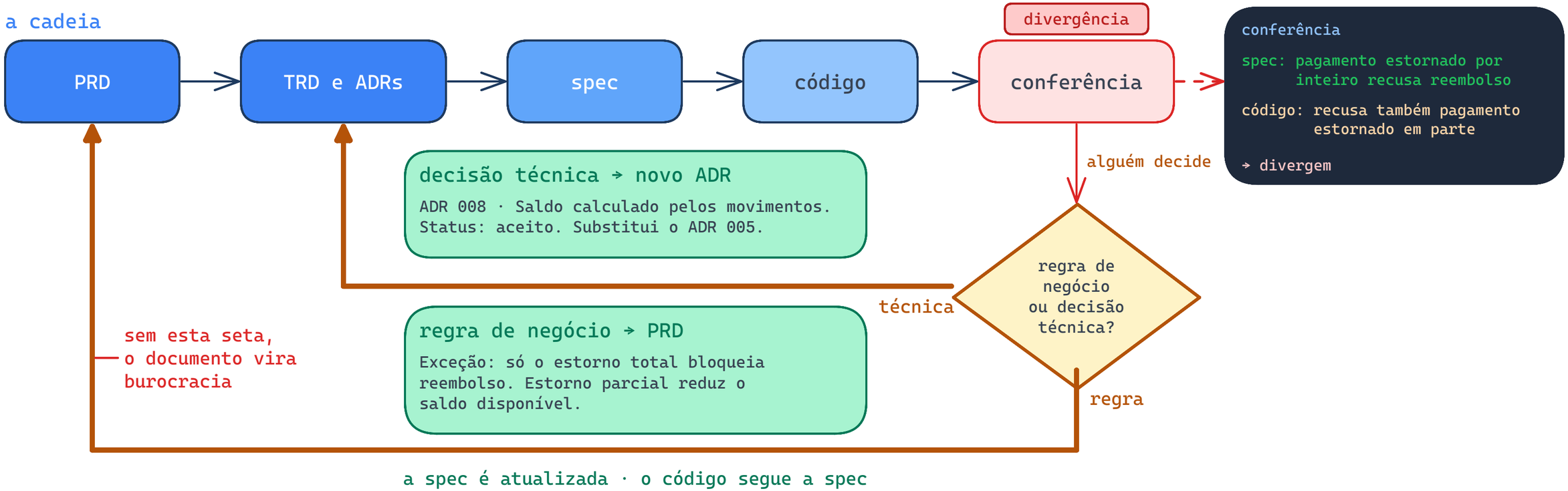
o que a spec carrega que o prompt não carrega

1. a decisão com a razão — na spec: Requisito e Razão
2. o critério de aceite verificável — na spec: Dado · Quando · Então
3. a fronteira do que está fora — na spec: Fora

no prompt: só o pedido, nenhum dos três

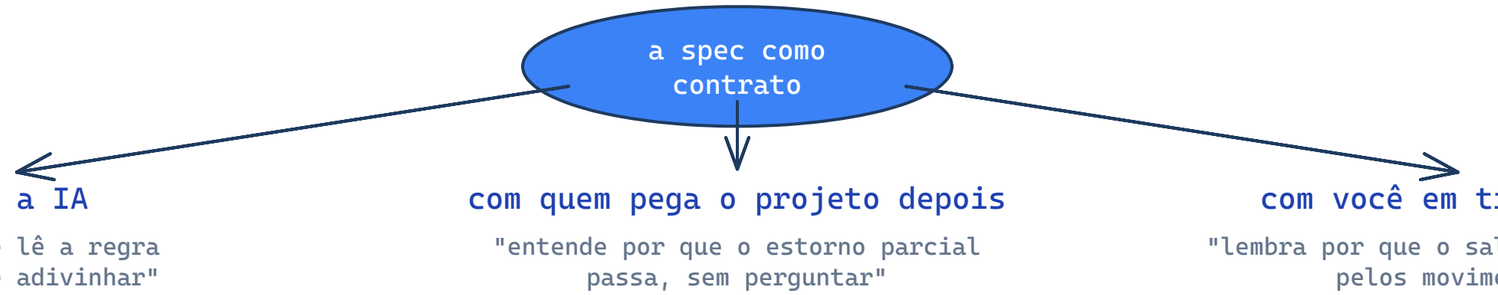
A decisão volta para o documento

spec e código discordam: alguém decide, e a decisão sobe para o documento certo



três contratos

o que essa cadeia sustenta



O OpenSpec no projeto

só pasta e markdown, versionados junto do código

```
npm install -g @fission-ai/openspec@1.10.0
openspec init
```

versão fixada, não latest

o init cria

seu projeto

init → openspec/ + comandos no agente

sem servidor, sem serviço à parte

Onde a especificação mora

```
openspec/
├── specs/          ← a fonte de verdade · nasce vazia
├── changes/        ← uma pasta por mudança proposta
└── config.yaml     ← o contexto do projeto · regras por artefato
```

config.yaml (trecho)

```yaml
schema: spec-driven

context: |
  Stack: Node.js, Express, PostgreSQL
  Testes: Jest

rules:
  proposal:
    - Sempre declarar o que ficou de fora
  specs:
    - Cenários em WHEN/THEN
```

— o que o agente sabe do projeto

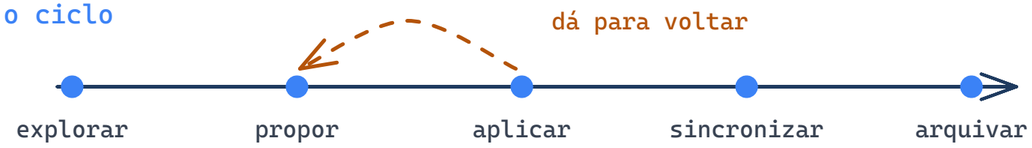
— regra própria de cada artefato

Os comandos do ciclo no agente

```
.agents/skills/
├── openspec-explore/           explorar: pensar sem escrever código
├── openspec-propose/           propor: nascem os artefatos
├── openspec-apply-change/      aplicar: implementa as tasks
├── openspec-sync-specs/        sincronizar: delta → specs/
└── openspec-archive-change/    arquivar: change → archive/
```

o caminho muda de ferramenta para ferramenta (ex.: .agents/skills/)

o ciclo



Características

- roda no agente de codificação que você já usa
- funciona em projeto que já existe, não só em projeto novo
- sem etapa travada: dá para voltar a um artefato
- leve: markdown versionado junto do código

A anatomia de um change

cada estação do ciclo deixa um arquivo para trás

exemplo único em todo o desenho
change bloquear-login-apos-tentativas · domínio auth

custo de corrigir um erro: barato aqui → caro na seguinte

revisão humana: o que se revisa aqui
as perguntas que ainda não têm resposta

revisão humana: o que se revisa aqui
o escopo, a alternativa descartada, o cenário

revisão humana: o que se revisa aqui
o código contra os cenários

revisão humana: o que se revisa aqui
o que entra na spec permanente (openspec/specs/)

1 · Explorar → 2 · Propor → 3 · Aplicar → 4 · Sincronizar e arquivar

propor produz os 4 arquivos

o delta vira spec permanente

nenhuma linha de código

só perguntas em aberto:

- bloqueia por quanto tempo?
- conta por usuário ou por IP?
- o usuário é avisado?

fora da pasta: explorar não deixa arquivo

proposal.md

```
## Por quê
Hoje o login aceita tentativas sem limite.

## O que muda
- Bloquear a conta por 15 minutos após
  5 tentativas erradas seguidas

## Fora do escopo
- Desbloqueio pelo suporte
```

tasks.md

```
## 1. Contador
- [ ] 1.1 Gravar tentativa errada
- [ ] 1.2 Zerar após login certo
## 2. Bloqueio
- [ ] 2.1 Recusar login bloqueado
```

tasks.md todo [x]

[x] 1.1 [x] 1.2 [x] 2.1

→ a lista acabou

×

cenário conferido √

→ o resultado foi conferido

antes: orienta a construção

depois: confere o resultado

specs/auth/spec.md — o spec delta

```
## ADDED Requirements

### Requirement: Bloqueio após tentativas
O sistema SHALL bloquear a conta após
5 tentativas erradas seguidas.

#### Scenario: Quinta tentativa errada
- WHEN o usuário erra a senha pela 5ª vez
- THEN a conta fica bloqueada por 15 minutos

#### Scenario: Login certo zera o contador
- WHEN o usuário acerta a senha
- THEN o contador de tentativas volta a zero
```

design.md

```
## Decisão
Contador de tentativas por usuário,
gravado no banco.

## Alternativa descartada
Contador em memória: some quando
a aplicação reinicia.
```

a pasta do change

```
openspec/changes/bloquear-login-apos-tentativas/
├── proposal.md
├── design.md
├── tasks.md
└── specs/
    └── auth/
        └── spec.md        ← o spec delta
```

como ler as setas

- propor produz o arquivo
- aplicar consome o arquivo
- sincronizar consome o arquivo

o critério de parada: escrito antes do código

o cenário nasce em propor, antes do código: aplicar constrói por ele e é conferido contra ele

Do change à spec permanente

nada é apagado: o delta entra na spec, a pasta vai para o arquivo

ANTES → SINCRONIZAR → ARQUIVAR

1 · ANTES

change pronto, tasks todas [x]

```
openspec/
├── specs/
│   └── auth/
│       └── spec.md   (ainda sem o bloqueio)
└── changes/
    └── bloquear-login-apos-tentativas/   −
        ├── proposal.md
        ├── design.md
        ├── tasks.md   todas [x]
        └── specs/auth/spec.md   ← o delta
```

− sai daqui inteira ao arquivar

2 · SINCRONIZAR

o delta se funde na spec principal

delta · changes/…/specs/auth/spec.md

```
## ADDED Requirements

### Requirement: Bloqueio após tentativas
O sistema SHALL bloquear a conta após
5 tentativas erradas seguidas.

#### Scenario: Quinta tentativa errada
- WHEN o usuário erra a senha pela 5ª vez
- THEN a conta fica bloqueada por
  15 minutos
```

descreve só o que muda

funde

spec principal · openspec/specs/auth/spec.md

```
# auth

### Requirement: Login com e-mail e senha
O sistema SHALL autenticar com e-mail
e senha.
  ...

+ ### Requirement: Bloqueio após tentativas
+ O sistema SHALL bloquear a conta após
+ 5 tentativas erradas seguidas.
+ #### Scenario: Quinta tentativa errada
+   ...
```

o requisito antigo fica · o novo entra

o que cada operação do delta faz

- **ADDED** → acrescenta
- **MODIFIED** → substitui a versão antiga
- **REMOVED** → tira
- **domínio novo** → arquivo de spec novo

3 · DEPOIS DE ARQUIVAR

nada some: tudo muda de lugar

```
openspec/
├── specs/
│   └── auth/
│       └── spec.md   + Bloqueio após tentativas
└── changes/   livre para o próximo
    └── archive/
        └── 2026-10-18-bloquear-login-apos-tentativas/   +
            ├── proposal.md
            ├── design.md
            ├── tasks.md
            └── specs/auth/spec.md
```

a pasta inteira muda de lugar, com o delta dentro
· changes/ fica livre para o próximo

ARQUIVAR: a pasta do change vai inteira para changes/archive/

openspec/specs/
→ como o sistema funciona hoje (a fonte de verdade)

openspec/changes/archive/
→ como ele chegou lá